7-1 · LangChain 챗봇 기초 · 실습 1.5H

ChatModel 호출과 프롬프트 템플릿 실습

첫 호출부터 체인 조립까지 — 부품이 하는 일을 확인

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① rag_env.py 하나로 고른 경로(OpenAI · Ollama)의 첫 호출에 성공한다

② 모델이 모르는 것을 일부러 물어 환각을 직접 본다

③ ChatPromptTemplate 의 {question} 빈칸을 채워 체인을 실행한다

④ 시스템 메시지와 temperature 가 답을 바꾸는 것을 비교한다

파일마다 cat > 파일.py << 'EOF' ... EOF 로 만들고 python 파일.py 로 실행합니다

실습 흐름

일곱 단계 — week7 폴더 하나에서 끝까지

모든 실습 파일이 rag_env.py 의 llm 을 불러 씁니다 — 경로는 그 한 파일(또는 .env)에서만 갈립니다

1. 환경 준비	2. 첫 호출	3. 모르는 것	4·5. 템플릿·체인	6·7. 역할·온도
venv · pip rag_env.py	call.py 토큰 수	unknown.py (일부러 실패)	template.py chain.py	system.py temp.py

1단계 — 환경 준비

핵심 명령 — 폴더 · 가상환경 · 설치 · 키

폴더	mkdir -p ~/data-eng/week7 · cd ~/data-eng/week7
가상환경	python3 -m venv .venv · source .venv/bin/activate
설치	pip install langchain langchain-openai langchain-ollama python-dotenv
확인	pip list | grep -i langchain → langchain 1.4.8 · core 1.7.4(실측)
키 (OpenAI 만)	echo 'OPENAI_API_KEY=sk-...' > .env · echo '.env' >> .gitignore

키를 코드에 직접 적지 않는다 — .env 에 두고 .gitignore 에 · 4-2 「이미지에 비밀값을 넣지 않는다」와 같은 이유

1단계 — rag_env.py

모델 설정 한 곳 — 이번 주 모든 실습이 불러 쓴다

1. **PROVIDER — 키가 있으면 openai, 없으면 ollama**
   .env 에 LLM_PROVIDER=ollama 를 넣으면 키가 있어도 Ollama

2. **get_llm() — gpt-4o-mini 또는 llama3.2**
   temperature 기본 0 · 파일 끝에서 llm = get_llm()

3. **get_emb() — 7-3 부터 쓴다**
   text-embedding-3-small(1536) 또는 embeddinggemma(768)

4. **python rag_env.py**
   PROVIDER = ollama · 모델 = llama3.2(실측) — 키를 넣었으면 openai · gpt-4o-mini

키가 있는데 ollama

.env 위치와

줄 모양 확인

2단계 — 첫 호출

call.py — 응답 객체 안을 들여다본다

1. **resp = llm.invoke("데이터 엔지니어링을 한 문장으로 소개해줘")**
   from rag_env import llm — 한국어 한 문장 답, 글은 resp.content 에

2. **print(type(resp))**
   <class 'langchain_core.messages.ai.AIMessage'>

3. **print(resp.usage_metadata)**
   {'input_tokens': 41, 'output_tokens': 64, 'total_tokens': 105}(실측 · Ollama)

4. **답에 「xử lý」 같은 다른 언어 낱말**
   작은 로컬 모델(3B)에서 흔하다 — 구조를 배우는 데는 지장 없음

첫 호출

모델을 올리느라

약 2분

3단계 — 일부러 실패시키기

unknown.py — 모델이 알 수 없는 것을 묻는다

이론에서 말한 한계를 직접 확인합니다 — 이번 주가 왜 필요한지 여기서 드러납니다

질문	실측 (Ollama)	모델에 따라	고치는 법
우리 회사 2026년 3분기 장애 건수는?	「정보는 제공되지 않았습니다」	그럴듯한 숫자 = 환각	우리 문서를 찾아 넣어 준다 — RAG

4·5단계 — 템플릿과 체인

빈칸 있는 틀을 만들고 파이프로 잇는다

ChatPromptTemplate	→	llm	→	StrOutputParser()
("system", 조교 역할) ("human", "{question}")		rag_env.py 에서 불러온 그대로		AIMessage 에서 글자만 꺼낸다

chain.py — chain = prompt | llm | StrOutputParser() → chain.invoke({"question": "ETL과 ELT의 차이를 설명해줘"})

template.py 는 모델을 부르지 않는다 — messages=[SystemMessage(...), HumanMessage(...)] 가 2초 안에 찍힌다(실측)

5단계 — 파서 유무 비교

파서를 빼면 — 글자와 AIMessage

prompt | llm | StrOutputParser()

.content 없이 바로 글 — ETL · ELT 세 문단(실측)

- isinstance(r2, str) → True
- type() 은 TextAccessor — str 을 물려받은 클래스
- 글자처럼 그대로 쓴다

VS

prompt | llm

결과는 AIMessage — .content 를 꺼내야

- type(r1).__name__ → AIMessage
- 파서는 「원하는 모양만 꺼내는」 부품
- 실측 답에 틀린 문장도 섞였다 — 7-5 의 이유

6단계 — 시스템 메시지

system.py — 「Kafka가 뭐야?」를 두 역할로

[면접관에게]

기술 용어로 3문장

- 실측 — Apache 의 분산 메시지 브로커
- 고가용성까지 맞게 답했다
- 같은 질문, 다른 말투와 깊이

VS

[초등학생에게]

비유로 두 문장 안에

- 실측 — 「Kafka는 미국의 작가였습니다」
- 소설가 카프카로 착각해 지어냈다
- RAG 에선 「찾은 내용 안에서만 답하라」

7단계 — temperature

temp.py — 0 과 1.0 으로 세 번씩

1. **for t in [0, 1.0]: m = get_llm(temperature=t)**
   같은 질문 「데이터 파이프라인을 한 문장으로」를 세 번씩
2. **temperature 0**
   세 번 중 둘이 글자까지 같았다 — 0 도 완전히 같지는 않다(실측)
3. **temperature 1.0**
   세 번 모두 달랐고 다른 언어 낱말이 더 많이 섞였다(실측)
4. **사실을 조회하는 용도라면**
   0 에 가깝게 두는 것이 안전

여섯 번 호출

Ollama 는
몇 분 걸린다

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. python rag_env.py 가 내 경로의 PROVIDER · 모델을 찍는다
2. call.py 에 한 문장 답 · AIMessage · 토큰 수 세 줄이 나온다
3. 모르는 것을 물으면 모른다고 하거나 지어낸다 — 후자가 환각
4. 파서가 있으면 글자(str), 없으면 AIMessage 가 나온다
5. 시스템 메시지로 말투가, temperature 로 답의 흔들림이 달라진다

7-1 · 실습 1.5H 마무리

다음은 대화 이력 챗봇 실습 (1H)

멀티턴 대화를 만들고, 기억이 어디에서 오는지 이력을 빼 보며 실험으로 확인합니다

- ✓ 이력 없이 두 번 물어 기억 없음 확인
- ✓ 메시지 목록을 함께 보내 이름 기억하기
- ✓ 콘솔 챗봇 chatbot.py 완성
- ✓ 이력이 길어질 때 토큰 증가와 자르기

가져갈 것

켜 둔 가상환경과 rag_env.py